

Note: wherever a package is mentioned below, it is optional — all classes may instead be placed together in a single .java file.

Exercise 1 — Car Registry with Serialization

Car Class

1. **Properties:**
 - carModel (String)
 - make (String)
 - year (int)
2. **Constructors:** a default constructor, and a parameterized constructor for carModel, make, and year.
3. **Getters and Setters** for each property.
4. **Serializable Interface:** implements Serializable so the object can be serialized and deserialized.
5. **toString() Override:** provides a meaningful string representation of the Car object.

CarManager Class

1. **Main Method:** the entry point for the program.
2. **List of Cars:** an ArrayList to store Car objects.
3. **Input Car Information:** use a loop to input information for at least three cars from the keyboard, via a method inputCarInfo().
4. **Car Model Validation:** validate the car model using a regular expression (a method isValidCarModel()), ensuring only uppercase letters and numbers are allowed.
5. **Serialize Cars:** serialize the list of Car objects and write them to a file named “cars.txt”, via a method serializeCars().
6. **Deserialize Cars:** read the information from “cars.txt” and deserialize it into a list of Car objects, via a method deserializeCars().
7. **Display Cars:** display the information of cars read from the file, via a method displayCars().

General notes: handle exceptions appropriately (IOException, NumberFormatException), provide clear prompts and error messages, and document the classes with comments.

Exercise 2 — Club Membership Registry with Serialization

Create a class named Member with the following properties:

```
private String memberID;  
private String memberName;  
private String address;
```

- Create 2 constructors.
- Create getter/setter methods for all properties.

- Override `toString()` to return a string with the class's property information.
- This class must implement the `Serializable` interface.

Create a class `ClubManager` to do the following:

- The memberID format is: ABBCCCCC where A is one of the characters in the set {T, V, A}, BB is one of {MB, MT, MN}, and C is a digit. Example: TMB12345, VMT12312 are valid.
- Declare a list of members using a generic `ArrayList`, and input information for all members of this list. The information should be entered from the keyboard using the `java.io` package (do not use the `Scanner` class).
- When entering information from the keyboard, use a regular expression to check that the memberID is valid. If it is not valid, display a message requiring the user to enter it again.
- Serialize (write) all members to a file named "member_of_club.txt".
- Read the information from "member_of_club.txt", store it in another list, and display it on screen.

Exercise 3 — Library Book Registry with Serialization

Class Book: should have the following private properties:

```
private String bookID;
private String title;
private String author;
```

Requirements: - Implement two constructors: one default constructor, and one constructor with all three parameters (bookID, title, author). - Implement getter and setter methods for each property. - Override `toString()` to return a string containing the book's ID, title, and author. - The class must implement the `Serializable` interface.

Class LibraryManager:

The bookID should follow this format: ABCXXXX, where: - A is a letter from the set {F, N, S}, representing Fiction, Non-fiction, or Science. - B is a letter from {M, H}, representing Morning shift or Afternoon shift of the librarian. - C is a letter from {P, D}, representing Paperback or Digital format. - XXXX is a 4-digit number.

Requirements: - Declare a list of books using `ArrayList<Book>`. - Input information for each book (bookID, title, author) from the keyboard, ensuring the bookID is validated with a regular expression. - Serialize the list of books to a file called "library_books.ser". - Read the serialized information from the file, store it in another list, and display it.

Additional constraint: if bookID is invalid, display an error message and request valid input again. Add a method that counts the number of books written by a particular author.

Exercise 4 — Employee Registry with Serialization

Create a class named `Employee` with the following private fields: - `employeeID` - `fullName` - `address` - `age` (int)

Requirements for the `Employee` class: - Create two constructors: a default one, and one with parameters to set all fields. - Provide getter and setter methods. - Override `toString()` to return the full information of the employee.

Create an `EmployeeTest` class to test the `Employee` class's functionality:

1. Input the information of 5 employees from the keyboard and add them to a `List<Employee>`. Use the `java.io` package to read input from the keyboard (do not use `Scanner`). Use generics to store the employee list.
2. Validate `employeeID` using a regular expression. The `employeeID` format should be: 2 letters, followed by 4 digits, and ending with 1 or 2 letters. For example, valid values are `AB1234C`, `XY5678D`, `ZZ0000EF`, while invalid values include `A1234`, `X12345G`, `XY123H1`.
3. Write the employee list to a file named "employee.dat".
4. Read the "employee.dat" file to retrieve the employee list and display it on the console.
5. From the employee list read from the file, display all employees with age greater than 25.

Exercise 5 — Student Records from a Text File

Create a class `Student` to describe personal information with the properties:

```
private String name;  
private int age;  
private String phone;
```

The class must have 2 constructors (one default, one parameterized), getter and setter methods. This class must be serializable.

Given a file `input.txt` containing information about individuals in the format:

```
Name  
Age  
Phone number  
-----
```

Write a program to read `input.txt` and build a list of `Student` objects (using generics). The data from the file must be validated: the name must be a string, the age must be a positive integer, and the phone number must be a string. Display an error message if there is invalid data, e.g. "Not a number" if the age is invalid.

Use `ObjectOutputStream` to write each object to a file named `Persons.obj`.

Exercise 6 — Personnel Management System with File Save/Load

Goal: Build a simple personnel management program in Java to manage employee information within a company.

Requirements:

1. Create an `Employee` class with the following properties:

- name (String)
- country (home region, String)
- birthYear (int)
- salary (int)

Create 2 constructors for the class: a no-argument constructor and a full-argument constructor.

2. Build the program with the following menu of functions:

Personnel Management Program

1. Enter employee information
2. Sort employees by salary, descending
3. Analyze by home region
4. Find employees by home region and minimum salary
5. Save the employee list to a file
6. Open the file and display the list
7. Exit

3. If the user selects an invalid option, ask them to enter it again.

4. Implement the following functions:

- **Enter employee information (Function 1):** allow the user to enter information for each employee, including name, home region, birth year, and salary. Use an ArrayList to store the entered employee list.
- **Sort employees by salary, descending (Function 2):** display the entered employee list sorted by salary in descending order.
- **Analyze by home region (Function 3):** count the number of employees per home region and display the results.
- **Find employees by home region and minimum salary (Function 4):** ask the user to enter a home region and a minimum salary, then display the list of employees whose home region matches and whose salary is at or above that minimum.
- **Save the employee list to a file (Function 5):** ask the user for an output file name, create the file in binary mode, and report an error if it cannot be created; otherwise write the employee list data to the file and confirm success.
- **Open the file and display the list (Function 6):** ask the user for a file name to open, in binary mode, and report an error if it cannot be opened; otherwise read the data from the file and display it on screen.
- **Exit (Function 7):** end the program.

Exercise 7 — Social Media Account Management with File Save/Load

Write a program that simulates social-media account management software, with the following requirements:

Data structure: - username (String) - password (String) - userType (String) - registerYear (Integer) - cookie (String) - price (Float)

Program header menu, shown when the program starts:

Account Management Program

1. Enter data | 2. Sort by username | 3. Sort by price | 4. Analyze by type | 5. Search by type and price | 6. Save to file | 7. Open file | 8. Exit

After the user selects a feature and it finishes running, except for the Exit option, the program should ask whether the user wants to continue:

Do you want to continue?

- Yes (press 'y' or 'Y')

- No (press 'n' or 'N')

- Clear the screen (press 'c' or 'C')

Your choice:

If the user enters an invalid choice, ask them to enter it again. If 'y', return to the menu to continue; if 'n', exit; if 'c', clear the screen.

Note: the program should require the user to enter account data before other options can be used; otherwise, it should display an error requiring the user to input account data first.

Entering the account list: if the user selects option 1: - Ask the user for the total number of accounts to enter. - Enter each account's information. - Validate the entered information: - 0 <= price <= 100,000 - The registration year must be within the last 5 years.

Option 2: display a table sorted alphabetically by username.

Option 3: display the list sorted by price in descending order.

Option 4: display a count of accounts grouped by userType.

Option 5: ask the user to enter a userType to search for and a minimum price; display all accounts matching that type with a price greater than the minimum.

Option 6 — Save to file: - Ask the user for the output file name. - Create the file in binary mode. - If the file cannot be created, return an error and display a message. - If successful, write the account list data currently held in memory to the file, and confirm success to the user.

Option 7 — Open file and display: - Ask the user for the file name to open. - Open the file in binary mode. - If the file cannot be opened, return an error and display a message. - Read the data from the file and display it on screen. - Close the file.

Option 8: exit the program.

(This is the final, most complete version of this exercise — it supersedes any earlier draft of the same account-management program that did not yet include file save/load.)

Exercise 8 — Library Book Records (File + Optional Database)

Develop a console application to manage books in a library with the following functions:

1. Add book information to the list.

2. Save the book list to a file.
3. Read the data from the file and display the book information on the console.
4. Search for a book by title or author.

Data to store: - **ID:** String - **Book Title:** String, at least 5 characters. - **Author:** String. - **Published Year:** integer, greater than 1900. - **Genre:** a list of genres (e.g. novel, science, literature...).

Requirements: - Create a test class that: - Declares a list of 5 books with information entered from the keyboard (use the `java.io` package instead of `Scanner`). - Validates the `bookTitle` and `publishedYear` fields. - Once all data has been entered, save it to a file `library_data.txt`. - Read the data from `library_data.txt` and display it on the console. - Allow searching for a book by title or author; if not found, display "NOT FOUND". - **Optional bonus:** create a database named `library` with a `books` table with columns for the corresponding book fields, and use a MySQL connector to insert the full book list into the table.

Exercise 9 — Inventory Records (File)

Develop a console application to manage warehouse inventory with the following requirements:

1. Add product information to the list.
2. Save the product list to a file.
3. Read the data from the file and display the product information on the console.
4. Search for a product by product name or category.

Data to store: - **Product ID (productId):** String. - **Product Name (productName):** String, at least 3 characters. - **Price (price):** decimal, greater than 0. - **Quantity (quantity):** integer. - **Category (category):** a list of categories (e.g. electronics, household, apparel...).

Requirements: - Create a test class that: - Initializes a list of 5 products with data entered from the keyboard (using `java.io`). - Validates `productName` and `price`. - Saves the product data to a file `inventory_data.txt`. - Reads and displays the data from `inventory_data.txt` on the console. - Allows searching for a product by name or category; if no match is found, displays "NOT FOUND".

Exercise 10 — Hotel Records (File)

Develop a console application to manage a list of hotels with the following requirements:

1. Add hotel information to the list.
2. Save the hotel list to a file.
3. Read the data from the file and display the hotel information on the console.
4. Search for a hotel by name or location.

Data to store: - **Hotel ID (hotelId):** String, must follow the format 123-ABC-456 / 235-GHI-352. - **Hotel Name (hotelName):** String, at least 3 characters. - **Address (address):** String. - **Room Price (roomPrice):** decimal, greater than 0. - **Star Rating (starRating):** integer from 1 to 5.

If any invalid input is encountered, ask the user to re-enter that field. The hotel ID must be unique — no duplicates allowed.

Requirements: - Create a test class that: - Initializes a list of 5 hotels with data entered from the keyboard (using `java.io`, not `Scanner`). - Validates `hotelName` and `roomPrice`. - Saves the hotel data to a file `hotel_data.txt`. - Reads and displays the data from `hotel_data.txt` on the console. - Allows searching for a hotel by name or location; if no match is found, displays “NOT FOUND”.

Exercise 11 — Employee Records from a Formatted Text File

Create a class `Employee` with the properties:

```
private String fullName;  
private int salary;  
private String department;
```

Requirements: 1. Create 2 constructors: a default constructor and a full-argument constructor. 2. Create getter and setter methods for all properties. 3. Override `toString()` to return the employee’s information. 4. This class must implement **serialization**.

Input file: given a file `employee_data.txt` containing employee information in the format:

```
Full Name  
Salary  
Department  
-----
```

Program requirements: 1. Read `employee_data.txt` and create a list of `Employee` objects (using generics). 2. Validate the data from the file: - Full name must be a non-empty string. - Salary must be a positive integer. - Department must be a non-empty string. 3. Display an error message if the data is invalid, for example: - “Salary is not a valid number” if the salary is invalid. - “Invalid department” if the department is empty. 4. Use `ObjectOutputStream` to write each object to a file named `Employees.obj`.

Exercise 12 — Book Records with ISBN Validation (File + Optional Database)

Create a bean class named `Book` to describe book information with the properties:

```
private String ISBN;  
private String title;  
private String author;  
private double price;
```

Requirements: 1. This class must have: - Two constructors: a default constructor, and a parameterized constructor to set the properties. - Getter and setter methods. - An override of `toString()` (inherited from `Object`) that returns the book’s information.

2. Create a class named `BookTest` to test the following:
 - Input 5 books from the keyboard and store them in a `List<Book>` (using generics).

- Validate the ISBN according to the format: 3 digits - 2 letters - 4 digits (e.g. 123-AB-4567 is valid, 12-ABC-456 is not). If invalid, ask the user to re-enter it.
- 3. Write the book list to a file named `books.dat`.
- 4. Read `books.dat` to retrieve the book list, then display it on the console.
- 5. From the list of books read, display information for all books priced under 50.
- 6. **Optional bonus:** create a database named `bookmanage` with a `books` table with columns for the corresponding fields; using a MySQL connector, write a method in `BookTest` to insert the full book list into the table.

Exercise 13 — Movie Records with Validation (File + Optional Database)

Create a class named `Movie` to describe movie information with the properties:

```
private String movieID;
private String title;
private String director;
private double rating;
```

Requirements: 1. This class must have: - Two constructors: a default constructor, and a parameterized constructor to set the properties. - Getter and setter methods. - An override of `toString()` that returns the movie's information.

- 2. Create a class named `MovieTest` to test the following:
 - Input 5 movies from the keyboard and store them in a `List<Movie>` (using generics).
 - Validate `movieID` according to the format: 2 uppercase letters - 4 digits (e.g. AB-1234 is valid; Ab-1234 or ABC-123 are not).
 - `title` must be at least 5 characters.
 - `rating` must be a number from 1 to 5.
 - If invalid, ask the user to re-enter.
- 3. Write the movie list to a file named `movies.dat`.
- 4. Read `movies.dat` to retrieve the movie list, then display it on the console.
- 5. From the list of movies read, display information for all movies with a rating greater than 4.
- 6. **Optional bonus:** create a database named `moviemanage` with a `movies` table with columns for the corresponding fields; using a MySQL Connector, write a method in `MovieTest` to insert the full list of `Movie` objects into the table.

Exercise 14 — Customer Records with Sorting

Create a class `Customer(int id, String name, String email, double balance)`.

Requirements: 1. Write getter and setter methods for the properties of this class. 2. Override `toString()` to return the information: id, name, email, and balance. 3. Implement the `Serializable` interface. 4. Perform the following: - Create a list containing **5 Customer objects**, with id and name passed through the constructor. The remaining data (email, balance) is entered from the keyboard. - Write the customer list to a file `customer.dat`

(using `ObjectOutputStream`). - Read the customer list from `customer.dat` and display the customers with a balance greater than 1,000. - Use `Collections.sort()` to sort the customer list by balance in descending order and display the sorted list.

Exercise 15 — Cosmetic Product Records with Comparable and Serialization

Creating the CosmeticProduct Class

Create a class named `CosmeticProduct` with the following private attributes: - `int productID` - `String productName` - `double price`

- Write public setter and getter methods for all attributes.
- Override the `toString()` method to return a string containing the cosmetic product's `productID`, `productName`, and `price` formatted appropriately.
- Implement the `Comparable<CosmeticProduct>` interface to allow sorting cosmetic products by price in ascending order.

Working with CosmeticProduct Objects and File I/O

- In your main method, create a `List` containing 3 `CosmeticProduct` objects.
 - The `CosmeticProduct` objects should be created by taking input from the keyboard.
 - Prompt the user to enter the `productID`, `productName`, and `price` for each product.
- Write the three `CosmeticProduct` objects to a file named `cosmetics.dat` using `ObjectOutputStream`.
 - Ensure that all necessary steps are taken to properly open, write to, and close the file, handling any possible exceptions.
- Read the three `CosmeticProduct` objects from the `cosmetics.dat` file into a new `List`.
 - Sort the list of products by price using the `Comparable` implementation.
 - Display the information of each product on the console in a readable format.

Exercise 16 — Freelancer Records with Validation

Create a Freelancer Class

Define a class `Freelancer` with the following fields, constructors, and getter/setter methods:

Field Name	Data Type	Description
<code>id</code>	<code>String</code>	Unique ID of the freelancer
<code>name</code>	<code>String</code>	Name of the freelancer
<code>hourlyRate</code>	<code>float</code>	Hourly rate in USD
<code>totalHoursWorked</code>	<code>int</code>	Total hours worked
<code>skills</code>	<code>List<String></code>	List of freelancer skills

Implementation Details: - Constructors: a no-argument constructor, and a constructor with all fields as arguments. - Getters and Setters: create getter and setter methods for all fields.

Validation Rules: - Name: must contain at least 3 characters. - Hourly Rate: must be greater than \$15. - Total Hours Worked: must be a non-negative integer.

Implement a Test Class

Implement the following functionalities in a `Test` class (all classes may be placed together in a single `.java` file):

1. **Add Freelancer:**
 - Input freelancer data from the keyboard using `BufferedReader` from the `java.io` package.
 - Validate the fields based on the rules above.
 - If validation fails, display appropriate error messages.
2. **Save Freelancers to File:**
 - Save all freelancer details to a file named `freelancers_data.dat`.
 - Use proper file handling methods to write data securely.
3. **Read and Display Freelancers from File:**
 - Read data from the `freelancers_data.dat` file and display the information in the console.
4. **Search Freelancers by Skill:**
 - Allow the user to input a skill (e.g. "ReactJS").
 - Display all freelancers possessing that skill, or display "NOT_FOUND" if no freelancers have the skill.